ОТЧЁТ ПО ДОМАШНЕЙ РАБОТЕ

Анализ вакансий разработчиков мобильных платформ

Дисциплина: Инженерия данных
Вариант № 9 — «Программист, разработчик на мобильных платформах»

Выполнила: Нугаева Камилла
ИСУ: 434674
Группа: J3312

Технологии: Python, RustFS (S3), DuckDB, Docker
Репозиторий: https://github.com/kamii86/hh-skill-analytics

Октябрь 2026 г.

1. Цель и задачи работы

Цель работы — построить аналитический пайплайн для исторического набора вакансий разработчиков мобильных платформ. Требуется организовать хранение исходного CSV в S3-совместимом RustFS, реализовать потоковый подсчёт частот навыков и сочетаний по схеме MapReduce, независимо проверить результаты запросами DuckDB, выполнить зарплатный и региональный анализ, а также исследовать производительность и восстановление после сбоя.

Реализованы три размера чанков (1 000, 5 000 и 10 000 строк), отдельная SQL-реализация, сверка всех комбинаций и контрольных сумм, анализ зарплатных интервалов, сравнение трёх городов, серия замеров времени и памяти и оптимизированный вариант обработки.

2. Исходные данные и подготовка

Исходный объект размещён по адресу s3://hh-lake/raw/Raw_Jobs.csv. Файл содержит 561 989 записей, размер — 1 556 363 336 байт. CSV имеет разделитель «;»; описание вакансии может содержать переносы строк внутри кавычек. Поэтому чтение выполняется стандартным CSV-парсером, а не разбиением по строкам.

Показатель	Значение
Записей в исходном файле	561 989
Отобранных уникальных вакансий	321
Из них с заполненными навыками	262
Повторных вакансий удалено	3
Исключений после ручного аудита	6

Отбор варианта № 9 выполнялся по названию должности: Android, iOS и кроссплатформенная мобильная разработка. Из анализа исключены вакансии других специальностей. После просмотра сомнительных описаний вручную удалены ещё шесть записей. Дубликаты выявлялись по идентификатору вакансии. Навыки приводились к нижнему регистру, очищались от краевых пробелов и дублирований внутри одной вакансии.

3. Архитектура обработки

Источник: RustFS предоставляет исходный CSV через S3 API. Первый независимый путь: Python MapReduce последовательно читает CSV чанками и строит частотные таблицы. Второй путь: DuckDB посредством httpfs выполняет SQL-расчёты непосредственно по объекту S3. Таблицы сравниваются по каждой комбинации. Отдельные скрипты формируют зарплатные и региональные таблицы, замеры производительности и отчёты проверки.

Схема MapReduce в работе реализована в одном процессе. Это потоковая модель map/reduce с ограничением рабочей памяти, а не распределённый вычислительный кластер.

4. Подсчёт навыков, пар и троек

На этапе map из списка навыков вакансии формируются одиночные элементы, неупорядоченные пары и тройки. Счётчики суммируются для всех обработанных чанков на этапе reduce. Каждая уникальная комбинация учитывается не более одного раза для одной вакансии.

Вид комбинации	Уникальных	Всего вхождений
Навыки	383	1 622
Пары	3 674	6 039
Тройки	16 440	19 142

Самые частые навыки: iOS — 106, Git — 104, Swift — 94, Android — 89, Kotlin — 78, ООП — 56, Java — 51, Android SDK — 38, Flutter — 34, Objective-C — 28.

5. Независимая проверка DuckDB

DuckDB независимо вычислял те же частоты с эквивалентной фильтрацией, обработкой дубликатов и нормализацией. Скрипт сверки сравнил все ключи и их частоты между двумя подходами.

Сравнение	Проверено	Расхождений
Одиночные навыки	383	0
Пары	3 674	0
Тройки	16 440	0
Всего	20 497	0

CSV-файлы MapReduce при размерах чанка 1 000, 5 000 и 10 000 оказались побайтово одинаковы по SHA-256. Точно так же совпали выходные файлы исходной и оптимизированной реализаций (skills, pairs, triples). Результаты проверок сохранены в каталоге verification/ RustFS.

6. Анализ заработной платы

Анализ ограничен предложениями в рублях: валюты не смешивались. В 45 вакансиях имеется полный диапазон «от X до Y RUR»; для него использована середина интервала. Для 112 вакансий известна нижняя граница, которая исследовалась отдельно. Сравнивались медианы по парам и тройкам навыков с медианой для наиболее частого одиночного навыка комбинации. Порог включения — минимум три зарплатных наблюдения.

Пример (середина интервала)	Наблюдений	Изменение медианы
iOS + Swift	13	+5 000 руб.
Android + Kotlin	5	+20 000 руб.
Java + Kotlin	4	−42 500 руб.
Dart + Flutter	5	−15 000 руб.

Это описательные сравнения, а не оценка причинного влияния навыка: выборки малы, данные могут различаться по стажу, должности, региону и другим признакам. Полные наборы CSV сохранены в salary/midpoint и salary/lower.

7. Региональный анализ

Сопоставлены Москва, Санкт-Петербург и Казань. Города извлекались из поля location; вакансии с неподходящим или отсутствующим регионом не включались.

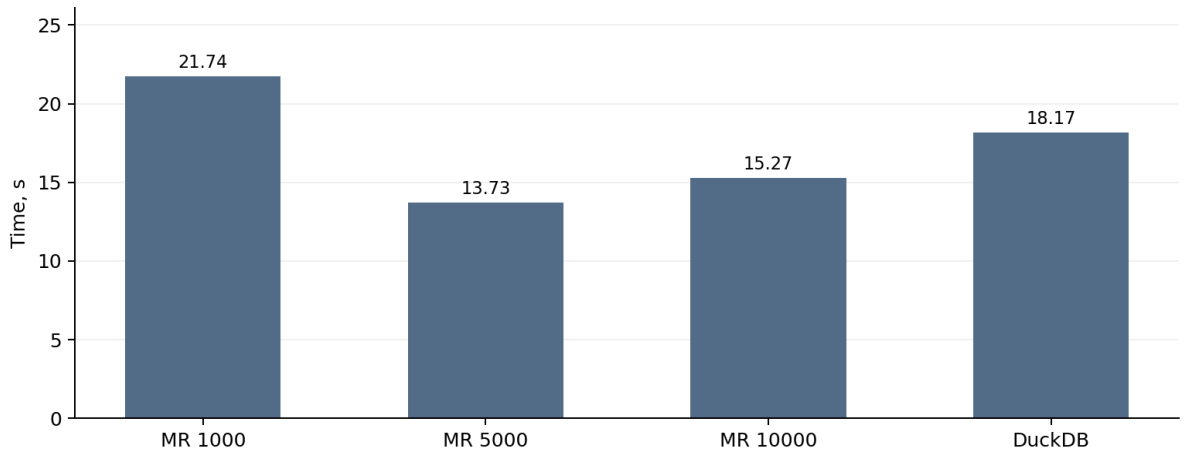
Показатель	Москва	Санкт-Петербург	Казань
Вакансий	49	22	5
С навыками	45	19	3
Нижняя граница зарплаты: n	17	7	4
Медиана нижней границы, руб.	70 000	140 000	85 000
Медиана середины интервала, руб.	92 500	170 000	145 000

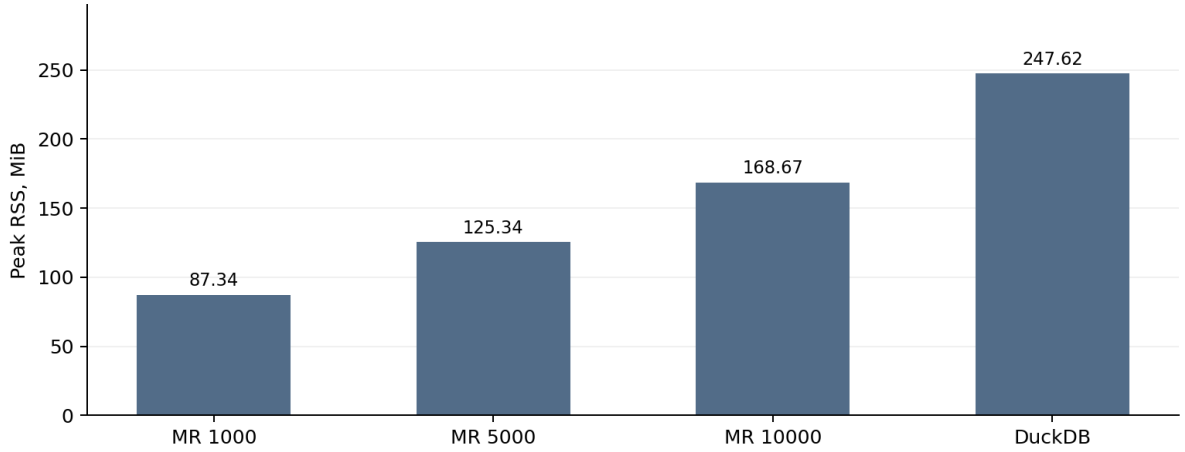
В Москве среди выбранных вакансий чаще всего указаны iOS и Swift, в Санкт-Петербурге навыки Android, Git, iOS и Swift имеют равную частоту — по шесть. По Казани число наблюдений особенно невелико; делать надёжные выводы о различиях региональных зарплат по этим данным нельзя.

8. Профилирование вычислений

Для каждой конфигурации выполнено три запуска. Измерялось полное время обработки и пиковое потребление памяти процесса (RSS). В таблице приведены средние значения трёх запусков, а не абсолютные максимумы.

Метод	Среднее время, с	Средняя пиковая RSS, МиБ
MapReduce, 1 000	21,744	87,34
MapReduce, 5 000	13,733	125,34
MapReduce, 10 000	15,270	168,67
DuckDB	18,169	247,62



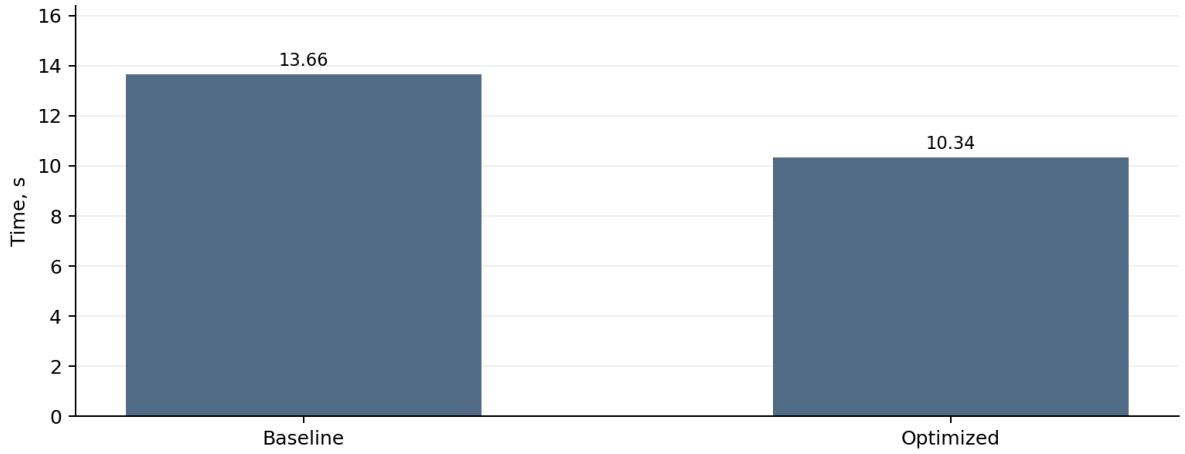


В этом эксперименте самым быстрым оказался вариант MapReduce с чанком 5 000 строк. Минимальный средний измеренный пик RSS наблюдался при чанке 1 000 строк. При выборе чанка необходимо учитывать компромисс времени и памяти. Замеры зависят от кэша и текущей нагрузки машины.

9. Бонусная оптимизация

В оптимизированной версии этап накопления счётчиков выполняется непосредственно в глобальные Counter без промежуточного счётчика и последующего объединения для каждого чанка. Проведено по три запуска исходной и оптимизированной версий при чанке 5 000; порядок запусков чередовался.

Показатель	Исходная	Оптимизированная
Среднее время, с	13,659	10,344
Средняя пиковая RSS, МиБ	125,68	123,74



Относительное сокращение среднего времени равно 24,27 %, снижение среднего измеренного пика RSS — 1,54 %. Отдельный запуск может оказаться медленнее из-за внешних факторов; вывод основан на серии измерений. Проверка SHA-256 подтвердила побайтовую идентичность результатов.

10. Проверка отказа и восстановления

Проведён контролируемый тест: маппер искусственно завершался исключением. Программа вернула ненулевой код, сохранённые ранее CSV в RustFS не изменились. Затем запущен полный повторный расчёт без искусственной ошибки; все три итоговых CSV получили исходные SHA-256. Это подтверждает устойчивость уже опубликованных результатов к выбранному сценарию сбоя и возможность пересчёта, но не означает автоматического восстановления с контрольной точки.

11. Воспроизводимость и расположение файлов

Основные команды запуска из корня репозитория:

```
python -m src.mapreduce_skills --chunk-size 5000
python -m src.duckdb_skills
python -m src.verify_results
python -m src.verify_chunk_sizes
python -m src.salary_analysis
python -m src.regional_analysis
python -m src.profile_runs
python -m src.benchmark_optimization
python -m src.test_mapper_failure
```

Код размещён в каталоге src/, графики — в results/figures/, CSV профилирования — в results/profiling/, данные сравнения оптимизаций — в results/optimization/. Полные выгрузки публикуются в RustFS по префиксам mapreduce/, duckdb/, verification/, salary/, regions/, profiling/, optimization/ и optimized/.

Для запуска необходим собственный доступ к работающему RustFS, параметры которого задаются в локальном .env по образцу .env.example. Исходный массив вакансий не включён в репозиторий.

12. Выводы

Построен сквозной процесс работы с данными в S3-совместимом хранилище: от фильтрации вакансий и потокового подсчёта навыков до независимой SQL-верификации, зарплатного и регионального анализа. MapReduce и DuckDB дали совпадение по всем 20 497 комбинациям. Проверки не обнаружили зависимости результата от размера чанка.

Экспериментально установлено, что для данной среды чанк 5 000 строк обеспечил наименьшее среднее время из трёх проверенных размеров. Бонусная оптимизация сократила среднее время на 24,27 % при сохранении идентичности выходных файлов. Тест искусственного сбоя подтвердил неизменность ранее записанных результатов и успешность повторного полного расчёта.

Ограничения: исторический датасет, неполные поля вакансий, небольшие подвыборки по зарплатам и регионам, а также локальный характер измерений производительности.

Приложение А. Объектное хранилище и проверка

Рисунок А.1 — Содержимое бакета hh-lake в RustFS

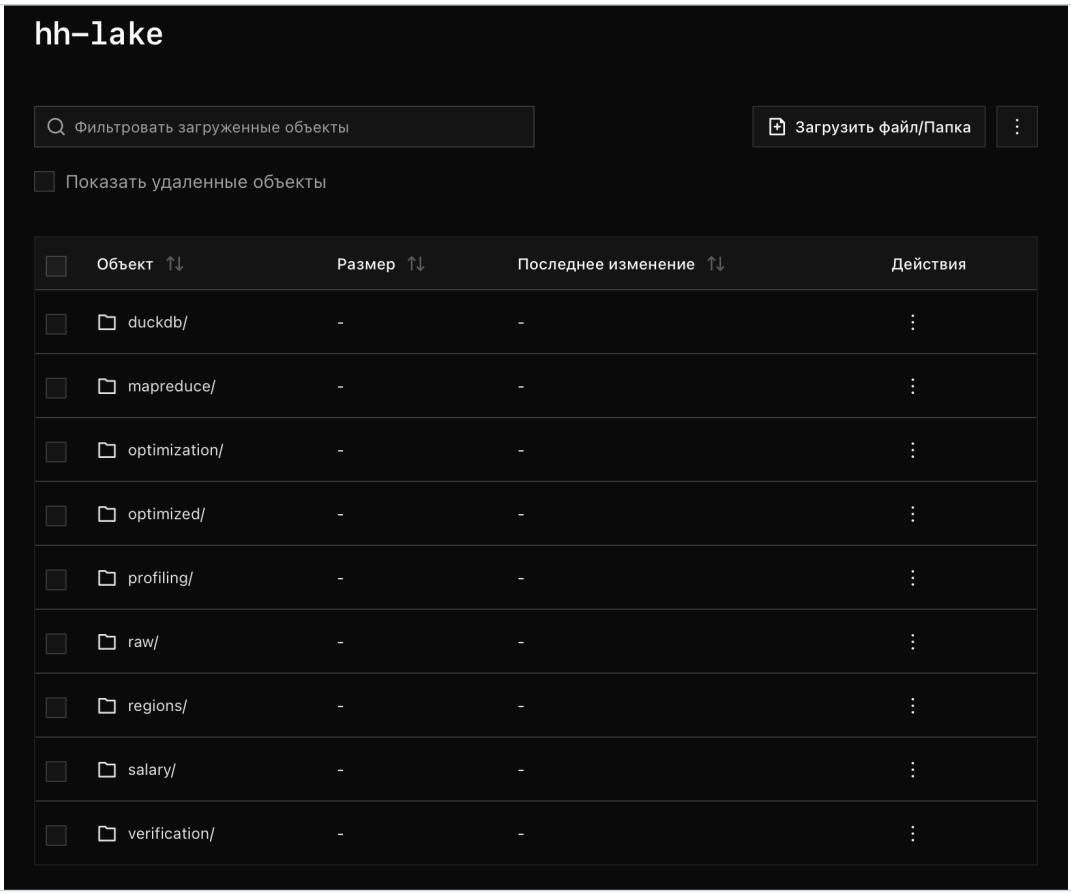
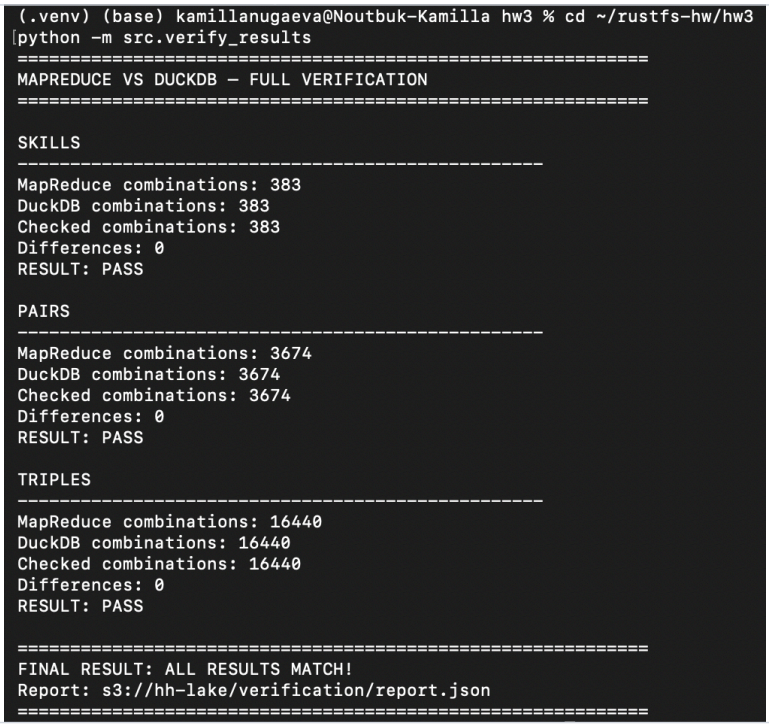


Рисунок А.2 — Сверка результатов MapReduce и DuckDB

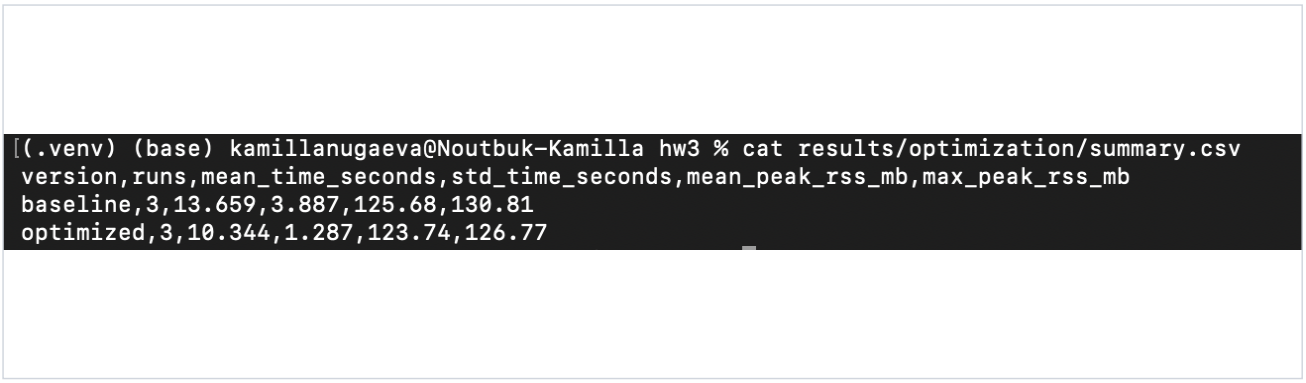


Приложение Б. Профилирование и оптимизация

Рисунок Б.1 — Результаты профилирования четырёх конфигураций



Рисунок Б.2 — Сравнение исходной и оптимизированной версий



По трём запускам каждой конфигурации наименьшее среднее время показал MapReduce с чанком 5000 строк (13,733 с). Оптимизированная реализация в отдельном сравнении сократила среднее время с 13,659 до 10,344 с (на 24,27%).

Показатели зависят от оборудования, состояния кэша и нагрузки системы. Представленные снимки подтверждают сохранённые CSV-результаты.

Приложение В. Тест отказа и восстановления

Рисунок В.1 — Отказ маппера и успешное восстановление результатов



Во время теста сбой приводил к аварийному завершению, но ранее записанные объекты RustFS оставались неизменными. Полный повторный запуск завершился успешно, контрольные суммы выходных файлов совпали.